



COMSATS University Islamabad, Abbottabad Campus

Department of Computer Science

Project Proposal
EVENT MANAGEMENT SYSTEM

DATABASE SYSTEM

Submitted on:

Group

Members:

Aiman Shabbir

(FA22-BSE-104)

Nimra Jadoon

(FA22-BSE-011)

Proposal for Event Management System ERD

1. Introduction:

The proposed **Event Management System** involves various entities that work together to facilitate the management of events, such as scheduling, tasks, budgeting, attendee management, vendors, feedback, and more. This system is designed to manage the details of each event and its associated processes, including handling attendees, vendors, tasks, resources, payments, and much more. The database structure is represented through an **Entity-Relationship Diagram (ERD)**, which clearly defines the entities, their attributes, and their relationships.

2. Entities and Attributes:

The following are the main entities involved in the Event Management System and their respective attributes:

1. User

- **user_id (PK)**: Unique identifier for the user.
- **username**: Name of the user.
- **email**: User's email address.
- **password**: User's password.
- **role**: Role of the user (e.g., admin, planner).

2. Event

- **event_id (PK)**: Unique identifier for the event.
- **event_name**: Name of the event.
- **date**: Date of the event.
- **location**: Location of the event.
- **organizer_id (FK)**: Foreign key referring to the user organizing the event.

3. Attendee

- **attendee_id (PK)**: Unique identifier for the attendee.
- **name**: Name of the attendee.

- **email:** Email of the attendee.
- **phone:** Contact number of the attendee.
- **event_id (FK):** Foreign key referring to the event the attendee is associated with.

4. Vendor

- **vendor_id (PK):** Unique identifier for the vendor.
- **name:** Name of the vendor.
- **service_type:** Type of service provided by the vendor (e.g., catering, decoration).
- **contact:** Contact person for the vendor.
- **email:** Email address of the vendor.

5. EventVendor

- **event_vendor_id (PK):** Unique identifier for the event-vendor relationship.
- **event_id (FK):** Foreign key referring to the event.
- **vendor_id (FK):** Foreign key referring to the vendor.
- **service_cost:** Cost of the service provided by the vendor.

6. Task

- **task_id (PK):** Unique identifier for the task.
- **task_name:** Name of the task.
- **status:** Current status of the task (e.g., pending, completed).
- **due_date:** Date by which the task should be completed.
- **event_id (FK):** Foreign key referring to the event to which the task belongs.

7. Budget

- **budget_id (PK):** Unique identifier for the budget.
- **event_id (FK):** Foreign key referring to the event.
- **total_budget:** Total allocated budget for the event.
- **spent:** Amount spent for the event so far.
- **remaining:** Remaining budget for the event.

8. Resource

- **resource_id (PK)**: Unique identifier for the resource.
- **resource_name**: Name of the resource.
- **quantity**: Quantity of the resource.
- **cost_per_unit**: Cost per unit of the resource.
- **event_id (FK)**: Foreign key referring to the event the resource is used for.

9. Payment

- **payment_id (PK)**: Unique identifier for the payment.
- **amount**: Amount paid.
- **date**: Date of the payment.
- **vendor_id (FK)**: Foreign key referring to the vendor who received the payment.
- **event_id (FK)**: Foreign key referring to the event the payment is associated with.

10. Feedback

- **feedback_id (PK)**: Unique identifier for the feedback.
- **event_id (FK)**: Foreign key referring to the event.
- **attendee_id (FK)**: Foreign key referring to the attendee who gave feedback.
- **rating**: Rating provided by the attendee (e.g., 1 to 5).
- **comments**: Additional comments from the attendee.

11. Sponsor

- **sponsor_id (PK)**: Unique identifier for the sponsor.
- **name**: Name of the sponsor.
- **email**: Email address of the sponsor.
- **phone**: Phone number of the sponsor.
- **event_id (FK)**: Foreign key referring to the event the sponsor is associated with.

12. Media

- **media_id (PK)**: Unique identifier for the media.
- **media_type**: Type of media (e.g., photo, video).

- **path**: File path for the media.
- **event_id (FK)**: Foreign key referring to the event the media is associated with.
- **uploaded_by (FK)**: Foreign key referring to the user who uploaded the media.

13. Invitation

- **invitation_id (PK)**: Unique identifier for the invitation.
- **event_id (FK)**: Foreign key referring to the event.
- **attendee_id (FK)**: Foreign key referring to the attendee who received the invitation.
- **status**: Status of the invitation (e.g., accepted, declined).

14. EventType

- **type_id (PK)**: Unique identifier for the event type.
- **type_name**: Name of the event type.
- **description**: Description of the event type.

15. EventSchedule

- **schedule_id (PK)**: Unique identifier for the event schedule.
- **event_id (FK)**: Foreign key referring to the event.
- **time_slot**: Time slot for the activity.
- **activity**: Activity description.
- **location**: Location where the activity will take place.
- **additional_notes**: Any additional notes related to the activity, such as equipment needed, special instructions, etc. (e.g., "Requires projector, catering, microphones").

16. ContactUs

- **contact_id (PK)**: Unique identifier for the contact form.
- **user_id (FK)**: Foreign key referring to the user who submitted the form.
- **message**: Message sent by the user.
- **response_status**: Status of the response to the message (e.g., pending, resolved).

3. Relationships Between Entities:

The relationships between entities are defined based on the actions and interactions that occur within the Event Management System. Below is a description of each relationship type:

1. User and Event

- **Type:** One-to-Many (1..*).
- **Relationship:** User "1" -- "0..*" Event : organizes
 - **Explanation:** A single user can organize multiple events, but an event must have exactly one user as its organizer.

2. User and ContactUs

- **Type:** One-to-Many (1..*).
- **Relationship:** User "0..*" -- "1" ContactUs : sends
 - **Explanation:** Each contact message must be sent by a single user, but a user can send multiple messages.

3. Event and EventType

- **Type:** One-to-One (1..1).
- **Relationship:** Event "1" -- "0..*" EventType : belongs_to
 - **Explanation:** An event belongs to one type (e.g., "conference" or "wedding"), and each type can have multiple events associated with it.

4. Event and Attendee

- **Type:** One-to-Many (1..*).
- **Relationship:** Event "1" -- "0..*" Attendee : includes
 - **Explanation:** An event can include multiple attendees, but each attendee is associated with only one event.

5. Attendee and Feedback

- **Type:** One-to-One (1..1).

- **Relationship:** Attendee "0..*" -- "1" Feedback : provides
 - **Explanation:** Each attendee can provide feedback about the event, and each feedback entry belongs to a single attendee.

6. Event and Task

- **Type:** One-to-Many (1..*).
- **Relationship:** Event "1" -- "0..*" Task : includes
 - **Explanation:** An event may have multiple associated tasks (e.g., "decorations" or "catering"), but a task belongs to a single event.

7. Event and Budget

- **Type:** One-to-One (1..1).
- **Relationship:** Event "1" -- "1" Budget : has
 - **Explanation:** Each event has one associated budget record, which tracks the total and remaining budget for the event.

8. Event and Resource

- **Type:** One-to-Many (1..*).
- **Relationship:** Event "1" -- "0..*" Resource : requires
 - **Explanation:** An event may require multiple resources (e.g., "chairs," "tables"), and each resource is associated with one event.

9. Event and Invitation

- **Type:** One-to-Many (1..*).
- **Relationship:** Event "1" -- "0..*" Invitation : sends
 - **Explanation:** An event sends invitations to multiple attendees, but an invitation belongs to a single event.

10. Event and EventSchedule

- **Type:** One-to-Many (1..*).
- **Relationship:** Event "1" -- "0..*" EventSchedule : follows

- **Explanation:** An event may follow multiple scheduled activities (e.g., "Opening Ceremony"), but a schedule entry belongs to only one event.

11. Event and Vendor

- **Type:** Many-to-Many (0..* - 0..*).
- **Relationship:** Event "0..*" -- "0..*" Vendor : collaborates
 - **Explanation:** Multiple vendors can collaborate on multiple events, such as catering services or decorators.

12. Event and Sponsor

- **Type:** Many-to-Many (0..* - 0..*).
- **Relationship:** Event "0..*" -- "0..*" Sponsor : supported_by
 - **Explanation:** Events may have multiple sponsors, and a sponsor can support multiple events.

13.Event and Media

- **Type:** One-to-Many (1..*).
- **Relationship:** Event "0..*" -- "0..*" Media : contains
 - **Explanation:** An event can have multiple media files (e.g., photos or videos), but each media entry is associated with a single event.

14. Event and Payment

- **Type:** One-to-Many (1..*).
- **Relationship:** Event "0..*" -- "0..*" Payment : processes
 - **Explanation:** An event can process multiple payments, but each payment is associated with a single event.

15.EventVendor and Vendor

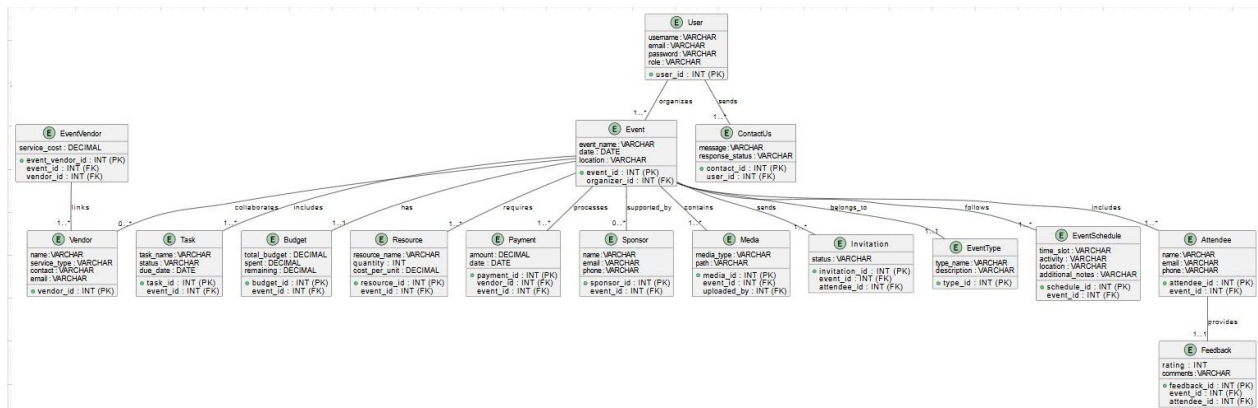
- **Type:** One-to-Many (1..*).
- **Relationship:** EventVendor "0..*" -- "1" Vendor : links

- **Explanation:** An intermediary table (EventVendor) tracks relationships between events and vendors, allowing multiple services per vendor to be tracked.

Summary of Relationship Types

Relationship Type	Example
One-to-One	Attendee provides Feedback.
One-to-Many	A User organizes multiple Events.
Many-to-Many	Multiple Vendors collaborate with multiple Events.

4. ERD DIAGRAM:



ERD Diagram Summary:

The **Entity-Relationship Diagram (ERD)** for the Event Management System is designed to model the relationships between the various entities involved in event planning. The primary focus is on managing events and their components, such as attendees, vendors, tasks, budgets, payments, and feedback.

□ Core Entities:

- **User:** Organizes events and can send contact messages.
- **Event:** Central entity connecting all other entities like attendees, tasks, budgets, schedules, etc.

- **EventType:** Categorizes events (e.g., conference, wedding).
- **Vendor and Sponsor:** Support events by providing services or funding.

□ **Key Relationships:**

- **One-to-Many:**
 - A User organizes multiple Events.
 - An Event includes multiple Attendees, Tasks, and Schedules.
 - An Attendee can provide Feedback for one event.
- **One-to-One:**
 - Each Event has one Budget.
- **Many-to-Many:**
 - Multiple Vendors collaborate on multiple Events.
 - Multiple Sponsors can support multiple Events.

□ **Supporting Entities:**

- **Resource:** Tracks items/events require (e.g., chairs, tables).
- **Media:** Stores media files (e.g., images, videos) related to events.
- **Payment:** Records financial transactions for events.
- **EventVendor:** Links events and vendors, tracking service costs.
- **ContactUs:** Allows users to send inquiries or messages.

□ **Purpose:**

- This ERD models an event management system, focusing on organizing events, managing attendees, resources, vendors, and finances.

Tables:

1. User Table

user_id	username	email	password	role
1	AhmedAli	ahmed.ali@pakistan.com	pass123	admin
2	FatimaKhan	fatima.khan@pakistan.com	secure123	planner
3	BilalSheikh	bilal.sheikh@pakistan.com	pakpass	user

2. Event Table

event_id	event_name	date	location	organizer_id
1	Eid Festival	2024-04-15	Karachi Expo Center	1
2	Tech Meetup	2024-05-10	Lahore IT Park	2
3	Book Fair	2024-06-20	Islamabad Convention Center	3

3. Attendee Table

attendee_id	name	email	phone	event_id
1	Ali Hassan	ali.hassan@pakistan.com	03001234567	1
2	Sara Ahmed	sara.ahmed@pakistan.com	03219876543	2
3	Zainab Noor	zainab.noor@pakistan.com	03324567890	3

4. Vendor Table

vendor_id	name	service_type	contact	email
1	PakCatering	Catering	Muhammad Rizwan	catering@pakcatering.com
2	DecorPak	Decoration	Sadia Malik	decor@decorpak.com
3	PakTechServices	Tech Support	Umar Farooq	tech@paktech.com

5. EventVendor Table

event_vendor_id	event_id	vendor_id	service_cost
1	1	1	50000
2	2	2	30000
3	3	3	20000

6. Task Table

task_id	task_name	status	due_date	event_id
1	Set up Stage	Pending	2024-04-10	1
2	Distribute Invitations	Completed	2024-05-01	2
3	Arrange Books	Pending	2024-06-15	3

7. Budget Table

budget_id	event_id	total_budget	spent	remaining
1	1	100000	50000	50000
2	2	80000	30000	50000
3	3	60000	20000	40000

8. Resource Table

resource_id	resource_name	quantity	cost_per_unit	event_id
1	Chairs	100	500	1
2	Projectors	5	10000	2
3	Bookshelves	10	2000	3

9. Payment Table

payment_id	amount	date	vendor_id	event_id
1	50000	2024-04-05	1	1
2	30000	2024-05-02	2	2

10. Feedback Table

feedback_id	event_id	attendee_id	rating	comments
1	1	1	5	Amazing event, well-organized!
2	2	2	4	Great tech talks, but seating was limited.
3	3	3	5	Excellent book collection!

11. Sponsor Table

sponsor_id	name	email	phone	event_id
1	HBL Bank	sponsor@hbl.com	03451234567	1
2	PTCL	sponsor@ptcl.com	03119876543	2
3	UBL Bank	sponsor@ubl.com	03224567890	3

12. Media Table

media_id	media_type	path	event_id	uploaded_by
1	Photo	/media/eid_festival.jpg	1	1
2	Video	/media/tech_meetup.mp4	2	2
3	Photo	/media/book_fair.jpg	3	3

13. Invitation Table

invitation_id	event_id	attendee_id	status	send_date
1	1	1	Sent	2024-04-01
2	2	2	Pending	2024-05-01
3	3	3	Sent	2024-06-10

14. EventType Table

type_id	type_name	description
1	Festival	Cultural or traditional festival event.
2	Meetup	Technology or professional networking event.
3	Expo	Exhibition or showcase of books or products.

15. EventSchedule Table

schedule_id	event_id	time_slot	activity	location	additional_notes
1	1	10:00-12:00	Opening Ceremony	Main Hall	Requires projector, catering, microphones
2	2	14:00-16:00	Keynote Speech	Conference Room	Speaker: John Doe, Video recording
3	3	11:00-13:00, 14:00-16:00	Book Launch, Panel Discussion	Auditorium	Author signings, Extra seating needed

16. ContactUs Table

contact_id	user_id	message	response_status
1	1	Need more details about the event schedule.	Resolved
2	2	Can we increase the budget?	Pending
3	3	Requesting additional vendor services.	Resolved

Merging and splitting

1. EventOrganizer (Event + User)

Event ID	Event Name	Date	Location	Organizer Name	Organizer Email
1	Eid Festival	2024-04-15	Karachi Expo Center	AhmedAli	ahmed.ali@pakistan.com
2	Tech Meetup	2024-05-10	Lahore IT Park	FatimaKhan	fatima.khan@pakistan.com
3	Book Fair	2024-06-20	Islamabad Convention Center	BilalSheikh	bilal.sheikh@pakistan.com

Anomalies:

- **Redundancy:** Event name, date, and location are repeated for each organizer. This causes redundancy if multiple events are organized by the same person.
- **Update Anomaly:** If the organizer's details change (e.g., name or email), it needs to be updated in multiple places, leading to inconsistency.
- **Deletion Anomaly:** If an event is deleted but the organizer still exists, you could lose information about the organizer if it's tied to that event.

Lossless or Lossy?

- **Lossy:** This is a **lossy** design because if we separate the event and organizer into separate tables (Event and User), we would still retain all necessary information. However, this design doesn't capture the normalized structure.

2. EventDetails (Event + EventType)

Event ID	Event Name	Date	Location	Event Type
1	Eid Festival	2024-04-15	Karachi Expo Center	Festival
2	Tech Meetup	2024-05-10	Lahore IT Park	Meetup
3	Book Fair	2024-06-20	Islamabad Convention Center	Expo

Anomalies:

- **Redundancy:** The event name, date, and location are repeated for each event type. If an event type is shared across multiple events, this leads to unnecessary repetition.
- **Update Anomaly:** If event details (e.g., date or location) change, it needs to be updated in every record where the event appears.

Lossless or Lossy?

- **Lossy:** This is a **lossy** design because the event details can be separated into an Event table and an EventType table, which would eliminate redundancy and make updates easier.

3. EventVendorDetails (Event + EventVendor)

Event ID	Event Name	Vendor Name	Vendor Service	Service Cost
1	Eid Festival	PakCatering	Catering	50000
2	Tech Meetup	DecorPak	Decoration	30000
3	Book Fair	PakTechServices	Tech Support	20000

Anomalies:

- **Redundancy:** Event name and details are repeated for each vendor.
- **Update Anomaly:** If the event name, location, or other details change, they must be updated in each row where the event is listed, leading to potential inconsistency.

- **Insertion Anomaly:** Adding a new vendor without associating it with an event would be difficult.

Lossless or Lossy?

- **Lossy:** This is a **lossy** design. A better approach would be to separate events and vendors into separate tables and use a many-to-many relationship.

4. EventSponsors (Event + Sponsor)

Event ID	Event Name	Sponsor Name	Sponsor Email	Sponsor Phone
1	Eid Festival	HBL Bank	sponsor@hbl.com	03451234567
2	Tech Meetup	PTCL	sponsor@ptcl.com	03119876543
3	Book Fair	UBL Bank	sponsor@ubl.com	03224567890

Anomalies:

- **Redundancy:** Event name and details are repeated for each sponsor.
- **Update Anomaly:** If event details change, they must be updated in every sponsor record.
- **Insertion Anomaly:** If a new sponsor is added but is not associated with an event, it will be hard to add that sponsor without emptying some fields.

Lossless or Lossy?

- **Lossy:** This is a **lossy** design because you could separate Event and Sponsor into two tables and use a foreign key to associate them.

5. EventFeedback (Event + Feedback)

Event ID	Event Name	Attendee Name	Rating	Comments
1	Eid Festival	Ali Hassan	5	Amazing event, well-organized!
2	Tech Meetup	Sara Ahmed	4	Great tech talks, but seating was limited.
3	Book Fair	Zainab Noor	5	Excellent book collection!

Anomalies:

- **Redundancy:** Event name and details are repeated for each feedback record.
- **Update Anomaly:** If event details change, it must be updated in each feedback record.
- **Deletion Anomaly:** Deleting an event could result in the loss of valuable feedback information unless the feedback is stored in a separate table.

Lossless or Lossy?

- **Lossy:** This is a **lossy** design. Separating Event and Feedback into two tables would remove redundancy and make updates easier.

6. EventMedia (Event + Media)

Event ID	Event Name	Media Type	Media Path
1	Eid Festival	Photo	/media/eid_festival.jpg
2	Tech Meetup	Video	/media/tech_meetup.mp4
3	Book Fair	Photo	/media/book_fair.jpg

Anomalies:

- **Redundancy:** Event name and details are repeated for each media entry.
- **Update Anomaly:** If event details change, they need to be updated in every media record where the event is mentioned.
- **Insertion Anomaly:** Adding new media without linking it to an event would be hard.

Lossless or Lossy?

- **Lossy:** This is a lossy design. A better approach would be to separate Event and Media into two tables, and link them with a foreign key.

Normalization (upto 3.5) and Decomposition

1. Task Table

Functional Dependencies:

1. $\text{task_id} \rightarrow \text{task_name}, \text{status}, \text{due_date}$
 - task_id determines task_name , status , and due_date .
2. $\text{event_id}, \text{task_id} \rightarrow \text{task_name}, \text{status}, \text{due_date}$
 - Combination of event_id and task_id determines the task attributes.

Candidate Key:

- **task_id** is the candidate key.

Prime Attributes:

- **task_id** is the prime attribute.

Non-prime Attributes:

- **task_name, status, due_date, event_id** are non-prime attributes.

1st Normal Form (1NF):

- All attributes contain atomic values, and there are no repeating groups.

Task Table in 1NF:

task_id	task_name	status	due_date	event_id
1	Task A	Pending	2024-12-31	101
2	Task B	Done	2024-12-31	102

2nd Normal Form (2NF):

- The table is in **1NF**, and all non-prime attributes are fully dependent on the entire candidate key (task_id), with no partial dependencies.

Task Table in 2NF:

task_id	task_name	status	due_date
1	Task A	Pending	2024-12-31
2	Task B	Done	2024-12-31

EventTask Table:

event_id	task_id
101	1
102	2

3rd Normal Form (3NF):

- The table is in **2NF**, and there are no transitive dependencies. Non-prime attributes (task_name, status, due_date) depend only on the candidate key (task_id).

Task Table in 3NF:

task_id	task_name	status	due_date
1	Task A	Pending	2024-12-31
2	Task B	Done	2024-12-31

EventTask Table:

event_id	task_id
101	1
102	2

3.5 Normal Form (3.5NF/BCNF):

- The table is in **3NF**, and there are no functional dependencies where non-prime attributes determine other non-prime attributes. Therefore, it is in **3.5NF** (or BCNF).
-

2. Resource Table

Functional Dependencies:

1. $\text{resource_id} \rightarrow \text{resource_name}, \text{quantity}, \text{cost_per_unit}$
 - resource_id determines resource_name , quantity , and cost_per_unit .
2. $\text{event_id}, \text{resource_id} \rightarrow \text{quantity}, \text{cost_per_unit}$
 - Combination of event_id and resource_id determines the quantity and cost_per_unit .

Candidate Key:

- **resource_id** is the candidate key.

Prime Attributes:

- **resource_id** is the prime attribute.

Non-prime Attributes:

- **resource_name, quantity, cost_per_unit, event_id** are non-prime attributes.

1st Normal Form (1NF):

- All attributes contain atomic values, and there are no repeating groups.

Resource Table in 1NF:

resource_id	resource_name	quantity	cost_per_unit	event_id
1	Resource A	10	100	101
2	Resource B	5	50	102

2nd Normal Form (2NF):

- The table is in **1NF**, and all non-prime attributes are fully dependent on the entire candidate key (resource_id), with no partial dependencies.

Resource Table in 2NF:

resource_id resource_name is_reusable

1	Resource A	Yes
2	Resource B	No

EventResource Table:

event_id resource_id quantity cost_per_unit

101	1	10	100
102	2	5	50

3rd Normal Form (3NF):

- The table is in **2NF**, and there are no transitive dependencies. Non-prime attributes depend only on the candidate key (resource_id).

Resource Table in 3NF:

resource_id resource_name is_reusable

1	Resource A	Yes
2	Resource B	No

EventResource Table:

event_id resource_id quantity cost_per_unit

101	1	10	100
102	2	5	50

3.5 Normal Form (3.5NF/BCNF):

- The table is in **3NF**, and there are no functional dependencies where non-prime attributes determine other non-prime attributes. Therefore, it is in **3.5NF** (or BCNF).
-

3. ContactUs Table

Functional Dependencies:

1. $\text{contact_id} \rightarrow \text{user_id}, \text{message}$
 - contact_id determines user_id and message .
2. $\text{contact_id} \rightarrow \text{response_status}$
 - contact_id determines response_status .

Candidate Key:

- **contact_id** is the candidate key.

Prime Attributes:

- **contact_id** is the prime attribute.

Non-prime Attributes:

- **user_id, message, response_status** are non-prime attributes.

1st Normal Form (1NF):

- All attributes contain atomic values, and there are no repeating groups.

ContactUs Table in 1NF:

contact_id user_id message response_status

1	101	Query 1	Pending
2	102	Query 2	Resolved

2nd Normal Form (2NF):

- The table is in **1NF**, and all non-prime attributes are fully dependent on the entire candidate key (contact_id), with no partial dependencies.

Contact Table in 2NF:

contact_id user_id message

1	101	Query 1
2	102	Query 2

Response Table:

response_id contact_id response_status response_message

1	1	Pending	No response yet
2	2	Resolved	Answered

3rd Normal Form (3NF):

- The table is in **2NF**, and there are no transitive dependencies. Non-prime attributes depend only on the candidate key (contact_id).

Contact Table in 3NF:

contact_id user_id message

1	101	Query 1
2	102	Query 2

Response Table:

response_id contact_id response_status response_message

1	1	Pending	No response yet
2	2	Resolved	Answered

3.5 Normal Form (3.5NF/BCNF):

- The table is in **3NF**, and there are no functional dependencies where non-prime attributes determine other non-prime attributes. Therefore, it is in **3.5NF** (or BCNF).
-

4. EventSchedule Table

1. Functional Dependencies:

In the **EventSchedule** table, the following functional dependencies can be identified:

- **schedule_id → event_id, time_slot, activity, location, additional_notes**
(schedule_id uniquely determines all other attributes in the table)
- **schedule_id → location**
(schedule_id determines the location, but location can be stored in a separate table for better normalization)
- **schedule_id → additional_notes**
(schedule_id determines additional_notes, which should be moved to a separate table to remove redundancy)

2. Candidate Key:

- The **candidate key** for the **EventSchedule** table is **schedule_id**, as it uniquely identifies each row.

3. Prime and Non-prime Attributes:

- **Prime Attributes:** These are attributes that are part of the candidate key.
 - **schedule_id** is the prime attribute.
- **Non-prime Attributes:** These are attributes that are not part of the candidate key.
 - **event_id, time_slot, activity, location, additional_notes** are non-prime attributes.

4. First Normal Form (1NF):

To achieve **1NF**, we need to ensure that each column contains atomic (indivisible) values, and there are no repeating groups. We also separate the rows for events that have multiple time slots.

Original Table (Unnormalized):

schedule_id	event_id	time_slot	activity	location	additional_notes
1	1	10:00-12:00	Opening Ceremony	Main Hall	Requires projector, catering, microphones
2	2	14:00-16:00	Keynote Speech	Conference Room	Speaker: John Doe, Video recording
3	3	11:00-13:00, 14:00-16:00	Book Launch, Panel Discussion	Auditorium	Author signings, Extra seating needed

Converted to 1NF (remove repeating groups and make each field atomic):

schedule_id	event_id	time_slot	activity	location	additional_notes
1	1	10:00-12:00	Opening Ceremony	Main Hall	Requires projector
1	1	10:00-12:00	Opening Ceremony	Main Hall	Catering
1	1	10:00-12:00	Opening Ceremony	Main Hall	Microphones
2	2	14:00-16:00	Keynote Speech	Conference Room	Speaker: John Doe
2	2	14:00-16:00	Keynote Speech	Conference Room	Video recording
3	3	11:00-13:00	Book Launch	Auditorium	Author signings
3	3	14:00-16:00	Panel Discussion	Auditorium	Extra seating needed

5. Second Normal Form (2NF):

To achieve **2NF**, we must ensure that:

1. The table is in **1NF**.
2. All non-key attributes are **fully functionally dependent** on the **entire** primary key.

Currently, **schedule_id** is the primary key, but **additional_notes** depends only on **schedule_id**, not on the entire composite key (event_id + time_slot). Thus, we need to move **additional_notes** to a separate table.

EventSchedule Table (2NF):

schedule_id	event_id	time_slot	activity	location_id
1	1	10:00-12:00	Opening Ceremony	1
2	2	14:00-16:00	Keynote Speech	2
3	3	11:00-13:00	Book Launch	3
3	3	14:00-16:00	Panel Discussion	3

ScheduleNotes Table (moved **additional_notes**):

note_id	schedule_id	note_text
1	1	Requires projector
2	1	Catering
3	1	Microphones
4	2	Speaker: John Doe
5	2	Video recording
6	3	Author signings
7	3	Extra seating needed

6. Third Normal Form (3NF):

To achieve **3NF**, we need to ensure that:

1. The table is in **2NF**.
2. There are no **transitive dependencies** (i.e., non-key attributes should not depend on other non-key attributes).

Here, **location** is a non-key attribute that depends on **schedule_id**. Since **location** can be reused across multiple schedules, we need to separate it into a **Location Table** and use a **location_id** in the **EventSchedule Table**.

EventSchedule Table (3NF):

schedule_id	event_id	time_slot	activity	location_id
1	1	10:00-12:00	Opening Ceremony	1
2	2	14:00-16:00	Keynote Speech	2
3	3	11:00-13:00	Book Launch	3
3	3	14:00-16:00	Panel Discussion	3

Location Table:

location_id	location
1	Main Hall
2	Conference Room
3	Auditorium

ScheduleNotes Table:

note_id	schedule_id	note_text
1	1	Requires projector
2	1	Catering
3	1	Microphones
4	2	Speaker: John Doe
5	2	Video recording

note_id	schedule_id	note_text
6	3	Author signings
7	3	Extra seating needed

7. Third Normal Form (3.5NF):

3.5NF is essentially a higher form of **3NF** that eliminates **join dependencies** and **transitive dependencies** more rigorously. This level is useful when there are **multivalued dependencies** (when a non-key attribute depends on another non-key attribute but not on the primary key). In this case, there are no multivalued dependencies, so the tables are already in **3.5NF**.

Final Summary:

Table Name Normalization Level

EventSchedule 3NF

ScheduleNotes 3NF

Location 3NF

This normalization process ensures that:

- Redundancy is minimized.
- Data consistency is maintained.
- Insertion, deletion, and update anomalies are avoided.

BUTTONS DETAILS

User Management

- **Add User:** Opens a form to input username, email, password, and role.
- **View Users:** Displays a list of users with user_id, username, email, and role.
- **Update User Details:** Opens a form prefilled with user_id, allowing updates to username, email, and role.

- **Delete User:** Deletes a user by username.
-

Event Management

- **Create Event:** Opens a form to input event_name, date, location, and organizer_id.
 - **View Events:** Displays a list of events with event_id, event_name, date, location, and organizer_id.
 - **Update Event Details:** Opens a form prefilled with event_id, allowing updates to event_name, date, and location.
 - **Delete Event:** Deletes an event by event_name.
-

Attendee Management

- **Add Attendee:** Opens a form to input name, email, phone, and event_id.
 - **View Attendees:** Displays a list of attendees with attendee_id, name, email, and event_id.
 - **Update Attendee Details:** Opens a form prefilled with attendee_id, allowing updates to name, email, and phone.
 - **Delete Attendee:** Deletes an attendee by attendee_id.
-

Vendor Management

- **Add Vendor:** Opens a form to input name, service_type, contact, and email.
 - **View Vendors:** Displays a list of vendors with vendor_id, name, service_type, and email.
 - **Update Vendor Details:** Opens a form prefilled with vendor_id, allowing updates to name, service_type, contact, and email.
 - **Delete Vendor:** Deletes a vendor by vendor_id.
-

Event-Vendor Management

- **Assign Vendor to Event:** Opens a form to link event_id and vendor_id, and input service_cost.
- **View Event Vendors:** Displays a list of event-vendor associations with event_vendor_id, event_id, vendor_id, and service_cost.
- **Update Vendor Assignment:** Opens a form prefilled with event_vendor_id to update service_cost.

- **Remove Vendor from Event:** Deletes an event-vendor association by event_vendor_id.
-

Task Management

- **Add Task:** Opens a form to input task_name, status, due_date, and event_id.
 - **View Tasks:** Displays a list of tasks with task_id, task_name, status, and due_date.
 - **Update Task Status:** Opens a form prefilled with task_id to update status.
 - **Delete Task:** Deletes a task by task_id.
-

Budget Management

- **Add Budget:** Opens a form to input event_id, total_budget, and spent.
 - **View Budgets:** Displays a list of budgets with budget_id, event_id, total_budget, spent, and remaining.
 - **Update Budget:** Opens a form prefilled with budget_id to update total_budget and spent.
 - **Delete Budget:** Deletes a budget by budget_id.
-

Resource Management

- **Add Resource:** Opens a form to input resource_name, quantity, cost_per_unit, and event_id.
 - **View Resources:** Displays a list of resources with resource_id, resource_name, quantity, and cost_per_unit.
 - **Update Resource Details:** Opens a form prefilled with resource_id to update resource_name, quantity, and cost_per_unit.
 - **Delete Resource:** Deletes a resource by resource_id.
-

Payment Management

- **Add Payment:** Opens a form to input amount, date, vendor_id, and event_id.
- **View Payments:** Displays a list of payments with payment_id, amount, date, vendor_id, and event_id.
- **Update Payment Details:** Opens a form prefilled with payment_id to update amount and date.

- **Delete Payment:** Deletes a payment by payment_id.
-

Feedback Management

- **Add Feedback:** Opens a form to input event_id, attendee_id, rating, and comments.
 - **View Feedback:** Displays a list of feedback with feedback_id, event_id, attendee_id, rating, and comments.
 - **Update Feedback:** Opens a form prefilled with feedback_id to update rating and comments.
 - **Delete Feedback:** Deletes feedback by feedback_id.
-

Sponsor Management

- **Add Sponsor:** Opens a form to input name, email, phone, and event_id.
 - **View Sponsors:** Displays a list of sponsors with sponsor_id, name, email, and event_id.
 - **Update Sponsor Details:** Opens a form prefilled with sponsor_id to update name, email, and phone.
 - **Delete Sponsor:** Deletes a sponsor by sponsor_id.
-

Media Management

- **Upload Media:** Opens a form to input media_type, path, event_id, and uploaded_by.
 - **View Media:** Displays a list of media with media_id, media_type, path, event_id, and uploaded_by.
 - **Update Media Details:** Opens a form prefilled with media_id to update media_type and path.
 - **Delete Media:** Deletes media by media_id.
-

Invitation Management

- **Send Invitation:** Opens a form to input event_id, attendee_id, and status.
- **View Invitations:** Displays a list of invitations with invitation_id, event_id, attendee_id, and status.
- **Update Invitation Status:** Opens a form prefilled with invitation_id to update status.

- **Delete Invitation:** Deletes an invitation by invitation_id.
-

Event Type Management

- **Add Event Type:** Opens a form to input type_name and description.
 - **View Event Types:** Displays a list of event types with type_id, type_name, and description.
 - **Update Event Type Details:** Opens a form prefilled with type_id to update type_name and description.
 - **Delete Event Type:** Deletes an event type by type_id.
-

Event Schedule Management

- **Add Schedule:** Opens a form to input event_id, time_slot, activity, and location.
 - **View Schedules:** Displays a list of schedules with schedule_id, event_id, time_slot, activity, and location.
 - **Update Schedule Details:** Opens a form prefilled with schedule_id to update time_slot, activity, and location.
 - **Delete Schedule:** Deletes a schedule by schedule_id.
-

ContactUs Management

- **View Messages:** Displays a list of messages with contact_id, user_id, message, and response_status.
 - **Respond to Message:** Opens a form prefilled with contact_id to update response_status.
 - **Delete Message:** Deletes a message by contact_id.
-

Navigation Buttons

- **Login:** Redirects to the main login with an overview of all entities.
- **Home:** Returns to the landing page of the application.
- **Logout:** Logs out the user and redirects to the login page.

Joins:

1. **Retrieve events along with their attendees and feedback.**

```
SELECT e.event_name, a.name AS attendee_name, f.rating, f.comments
FROM Event e
JOIN Attendee a ON e.event_id = a.event_id
JOIN Feedback f ON e.event_id = f.event_id AND a.attendee_id = f.attendee_id;
```

2. Get a list of events along with the vendors and their service costs.

```
SELECT e.event_name, v.name AS vendor_name, ev.service_cost
FROM Event e
JOIN EventVendor ev ON e.event_id = ev.event_id
JOIN Vendor v ON ev.vendor_id = v.vendor_id;
```

3. Retrieve events with their organizer details and budget.

```
SELECT e.event_name, u.username AS organizer, b.total_budget, b.spent, b.remaining
FROM Event e
JOIN User u ON e.organizer_id = u.user_id
JOIN Budget b ON e.event_id = b.event_id;
```

Nested Queries

2.1. Nested Query: Get events where the total budget is greater than the amount spent.

```
SELECT event_name
FROM Event
WHERE event_id IN (
    SELECT event_id
    FROM Budget
```

WHERE total_budget > spent

);

2.2. Nested Query: Get the attendees who provided a rating of 5 for events in Lahore.

SELECT name

FROM Attendee

WHERE attendee_id IN (

SELECT attendee_id

FROM Feedback

WHERE rating = 5 AND event_id IN (

SELECT event_id

FROM Event

WHERE location = 'Lahore'

)

);

2.3. Nested Query: Find the event names that have vendors with a service cost greater than 40,000.

SELECT event_name

FROM Event

WHERE event_id IN (

SELECT event_id

FROM EventVendor

WHERE service_cost > 40000

);

3.1. Aggregation Query: Get the total number of attendees for each event.

SELECT e.event_name, COUNT(a.attendee_id) AS total_attendees

FROM Event e

LEFT JOIN Attendee a ON e.event_id = a.event_id

GROUP BY e.event_name;

3.2. Aggregation Query: Get the average rating for each event.

```
SELECT e.event_name, AVG(f.rating) AS average_rating
FROM Event e
LEFT JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.event_name;
```

3.3. Aggregation Query: Get the total amount paid to each vendor.

```
SELECT v.name AS vendor_name, SUM(p.amount) AS total_paid
FROM Vendor v
JOIN Payment p ON v.vendor_id = p.vendor_id
GROUP BY v.name;
```